

Week 5 — Tokens, Stablecoins, Replay Attacks, UTXO, and Bitcoin Transaction Execution

1) What is a token?

The lecture starts with a very simple definition:

A **token** is a digital unit of value recorded on a blockchain. It is:

- **not** a coin with its own blockchain
- but a programmable asset that lives on top of an existing blockchain.

This is a very important distinction because students often mix up **coins** and **tokens**.

1.1 What does “digital unit of value” mean?

It means a token is some on-chain thing that the blockchain tracks as having value, function, or meaning.

That value could mean:

- money,
- access,
- rights,
- ownership,
- loyalty points,
- shares,
- or something else.

So token does not automatically mean “currency only.”

1.2 Why is it not a coin?

Because a token does not run its own blockchain from scratch.

For example:

- Bitcoin is a coin on the Bitcoin blockchain

- Ether is a coin on Ethereum
- but a token like USDC on Ethereum is not the native coin of Ethereum

It lives **on top of** Ethereum's blockchain.

1.3 Why call it programmable?

Because token rules are usually implemented through a **smart contract**.

That means the token's behavior can be coded:

- who can hold it,
- how it transfers,
- when it can be minted or burned,
- what rights it represents.

So a token is not just a number — it is a coded asset.

2) Coins vs tokens

The next slide directly compares them.

Coin

- has its own blockchain
- used to pay transaction fees
- examples: Bitcoin, Ether

Token

- built on top of another blockchain
- uses that blockchain for security
- created via smart contracts

This is one of the simplest and most likely short questions.

2.1 Coin

A coin is native to its blockchain.

Example:

- BTC belongs to Bitcoin

- ETH belongs to Ethereum

These are the main assets of their own chains.

They are usually also used to pay network fees.

2.2 Token

A token depends on an existing blockchain.

It does not create its own miner/validator network from scratch.
Instead, it uses the host chain's infrastructure.

Example:

a stablecoin on Ethereum uses Ethereum's blockchain as the environment in which it exists.

2.3 Main difference in one line

Coin = native asset of a blockchain

Token = asset built on top of an existing blockchain

That is the clean memory line.

3) What can a token represent?

The lecture says a token can represent:

- money (stablecoins)
- voting rights
- access rights
- ownership shares
- loyalty points
- real-world assets

And then gives the key sentence:

It's just a programmable entry in a distributed ledger.

This sentence is very deep.

3.1 Why is this so important?

Because it means blockchain value is not limited to "coins."

A token is really an on-ledger record with rules.

If the rules say it represents:

- a dollar claim,
- a hotel share,
- a governance vote,
- or event access,

then that is what it becomes in practical use.

3.2 “Programmable entry” means what?

It means the ledger stores not only balances, but also the **logic** of what those balances mean and how they behave.

So tokens are flexible.

3.3 Real-world assets

This is where blockchain becomes broader than currency.

A token could represent:

- a share in a company,
- a gram of gold,
- a piece of real estate,
- a reward point system.

So the lecture is broadening your understanding of blockchain utility.

4) How does a token work?

The lecture says that on smart-contract blockchains like Ethereum:

- a smart contract defines the token rules
- the ledger records balances
- transfers are just state updates
- stablecoins are tokens designed to represent fiat money.

This slide is very important because it gives the practical mechanics.

4.1 Smart contract defines token rules

The token is not floating magically. A smart contract contains the logic.

That contract may define:

- total supply,
- who can mint,
- who can burn,
- transfer rules,
- balances,
- permissions.

4.2 Ledger records balances

Once the smart contract exists, the blockchain ledger keeps track of who owns how many tokens.

So if your address holds 100 units, that is just recorded state.

4.3 Transfers are state updates

This is a key idea.

When you “send” a token, the blockchain is really updating state:

- subtract from sender
- add to receiver

So token movement is really ledger state transition.

4.4 Stablecoins as tokens

A stablecoin is just a token whose intended meaning is:
“this on-chain unit should behave like fiat value.”

That is why the lecture says stablecoins are simply tokens designed to represent fiat money.

5) Stablecoins — an alternative blockchain use case

The lecture says stablecoins are an alternative blockchain use case, and asks: transactions involving stablecoins happen on which blockchain? It reminds that transactions must be written in a ledger to have a record.

This is an important conceptual shift.

5.1 Why “alternative use case”?

Because many people think blockchain = volatile cryptocurrencies like BTC or ETH.

But stablecoins show another use:

bringing fiat-like value into blockchain systems.

5.2 On which blockchain do stablecoin transactions happen?

On the blockchain where the stablecoin token exists.

For example, if a stablecoin is issued on Ethereum, then its transfers are Ethereum ledger entries.

So stablecoin transaction history still needs a blockchain ledger.

5.3 Why is the ledger important?

Because without a ledger:

- no one knows balances,
- no one knows transfer history,
- no record exists of ownership change.

So even stablecoins need blockchain bookkeeping.

6) USDC / USDT — how stablecoins stay linked to USD

The lecture explains a stablecoin like USDC or USDT this way:

Step 1 — Issuance

- you send \$1 to the issuer
- issuer mints 1 stablecoin
- that \$1 is held in reserves

Step 2 — Redemption

- you return 1 stablecoin
- issuer burns it
- you receive \$1

This convertibility keeps the token price anchored.

This is one of the most important Week 5 concepts.

6.1 What does “1 token = 1 USD” really mean?

It means the issuer promises redeemability.

The issuer is saying:

“If you bring back this token, we will give you one real dollar (or the equivalent reserve-backed redemption value).”

6.2 Why do reserves matter?

Because the promise only works if real backing exists.

The lecture says reserves may include:

- bank deposits
- T-bills
- etc.

So the token is not stable just because people hope it is stable.

It is stable because:

- there is a redemption promise
- and reserves are supposed to back it

6.3 Minting and burning

This is very important.

When dollars go in, tokens are minted.

When tokens come back for redemption, they are burned.

So supply expands and contracts with issuance/redemption.

6.4 Why does this keep the peg?

Because if redemption is trusted and functional, the market knows the token should be worth about one dollar.

That keeps price anchored near \$1.

7) Arbitrage — why stablecoin price stays near \$1

The lecture says:

If stablecoin trades above \$1 (say \$1.02):

- traders buy from issuer for \$1
- sell for \$1.02
- price gets pushed down

If stablecoin trades below \$1 (say \$0.98):

- traders buy cheap
- redeem for \$1
- price gets pushed up

This arbitrage keeps it close to \$1.

This is a very clean economic idea.

7.1 What is arbitrage?

Arbitrage means exploiting a price difference for profit.

If the same asset is “supposed” to be worth \$1 but trades at a different price, traders profit by correcting that mismatch.

7.2 Above \$1 case

Suppose market price is \$1.02.

A trader can:

- get new tokens from issuer for \$1 each
- sell them in market for \$1.02
- earn 2 cents per token

As many traders do this, supply in the market increases, and price falls toward \$1.

7.3 Below \$1 case

Suppose market price is \$0.98.

A trader can:

- buy the token cheaply in market
- redeem it for \$1
- earn 2 cents per token

As many traders do this, market demand for the underpriced token rises, and price moves back up.

7.4 Main point

Stablecoin stability is not magic.
It is supported by:

- reserves,
 - redemption,
 - and arbitrage behavior.
-

8) ARTs versus EMTs

The lecture includes a table on page 10 comparing **Asset-Referenced Tokens (ARTs)** and **Electronic Money Tokens (EMTs)**. The table says:

- ARTs are pegged to multiple assets like fiat, commodities, or crypto
- EMTs are pegged to a single fiat currency like USD or EUR
- ARTs are more for investment/store of value
- EMTs are more for payments/transactions
- examples of ARTs: Diem, tokenized gold
- examples of EMTs: USDT, USDC, EUROe
- ART reserves must match referenced assets
- EMT reserves must be in the same fiat currency
- ART issuers are crypto firms/blockchain projects
- EMT issuers are licensed Electronic Money Institutions.

This table is visually important on page 10, and sir could definitely ask it.

8.1 ART

Asset-Referenced Token means the token's value references a basket or mix of assets.

For example:

- maybe some fiat
- some commodities
- some crypto

So it is broader and not tied to just one fiat currency.

8.2 EMT

Electronic Money Token is linked to **one fiat currency**.

So an EMT is more like:

- 1 token = 1 USD
- or
- 1 token = 1 EUR

This is more payment-oriented.

8.3 Main difference

ART = references multiple asset types

EMT = references one fiat currency

That is the key memory line.

Part 2 — Bitcoin Transactions

Now the lecture changes topic and becomes much more technical.

The outline includes:

- replay attacks
- UTXO model
- Bitcoin transaction format
- stack-based transaction execution
- legacy transactions
- transaction malleability
- segwit
- Merkle trees.

From the pages you shared here, the detailed explanation mainly covers:

- replay attacks
- UTXO
- legacy transaction format
- ScriptSig / ScriptPubKey
- P2PKH
- execution on the stack

So I'll explain those deeply.

9) What information is needed in a financial transaction?

The lecture first asks what information must be included in a transaction:

- payer
- payee
- amount
- authorization

Then it refines the authorization from:

SigAlia(Payer || Payee || Amount)

to

SigAlia(Hash(Payer || Payee || Amount))

This links directly to Week 3.

9.1 Basic transaction elements

Any payment system needs to know:

- who is sending
- who is receiving
- how much
- proof that sender authorized it

Without these, a payment record is incomplete.

9.2 Why sign the hash, not the raw message?

Because hashing before signing is more secure and efficient.

Instead of signing the full message directly, the sender signs its digest.

That gives integrity and authentication in a cleaner way.

9.3 This is the conceptual message format

So the signed meaning is:

“Alia authorizes transfer of this amount to Babar.”

That is the heart of digital transaction authorization.

10) Account-based model

The lecture then shows the **account-based model**:

- store list of accounts and balances
- transaction valid if enough balance exists
- payer debited, payee credited
- Ethereum uses account-based model.

This is very easy to understand because it resembles banking.

10.1 How account-based systems work

You keep a list like:

- Alia: 100,000
- Babar: 0

If Alia sends 50,000:

- Alia becomes 50,000
- Babar becomes 50,000

So the ledger tracks balances directly.

10.2 Why is this intuitive?

Because this is how normal bank accounts feel.

You think in terms of account balance, not in terms of “coins from earlier transactions.”

10.3 Ethereum uses this

That is an important detail. Ethereum is taught here as using the account-based model.

11) Replay attack in the account-based model

The lecture then shows a replay attack:

the same transaction from Alia to Babar for PKR 50,000, with the same authorization, is reused again, causing Alia's balance to go from 100,000 $\rightarrow$ 50,000 $\rightarrow$ 0 and Babar from 0 $\rightarrow$ 50,000 $\rightarrow$ 100,000.

This is a very important problem.

11.1 What is replay?

Replay means reusing a valid old signed transaction again.

If the network accepts the same transaction twice, then the payer loses money twice.

11.2 Why does this happen?

Because the signature is valid for that transaction content.

If nothing makes the transaction unique per use, then the same signed message may appear reusable.

11.3 Why is this dangerous?

Because a valid past message should not remain eternally re-executable.

So the system must distinguish:

- original intended spend
- malicious repeated replay

12) Preventing replay attacks using SEQ / nonce-like identifiers

The lecture says replay attacks can be prevented by associating a unique identifier **SEQ** with each transaction:

- randomly generated when transaction is created
- payer signs $m = (\text{SEQ} || \text{Payer} || \text{Payee} || \text{Amount})$
- system checks that the same SEQ is not reused
- Ethereum prevents replay attacks by tracking SEQ per user
- Bitcoin instead uses a different model: UTXO.

12.1 Why SEQ helps

Because even if payer, payee, and amount are identical, the transaction becomes unique through SEQ.

So replaying the exact same signed data again is detectable.

12.2 What does the system do?

It remembers which identifiers were already used.

If the same SEQ appears again, the system rejects it.

12.3 Ethereum vs Bitcoin

The lecture uses this to contrast:

- Ethereum: account-based + sequence/nonces
- Bitcoin: different model entirely, namely UTXO

That contrast is important.

13) Bitcoin uses the UTXO model

The lecture says:

- a bitcoin can only be spent once
- whenever a bitcoin is spent, new coins are generated
- a Bitcoin transaction consumes an unspent coin and generates new unspent coins.

This is one of the most important technical ideas in Bitcoin.

13.1 What is UTXO?

UTXO = **Unspent Transaction Output**

It means Bitcoin does not mainly think in terms of account balances.

Instead, it thinks in terms of **spendable outputs from earlier transactions**.

13.2 “A bitcoin can only be spent once”

This means each UTXO, once consumed as input, is gone.

It cannot be reused again.

That is how Bitcoin naturally prevents replay/double-spending in its own model.

13.3 “Whenever a bitcoin is spent, new coins are generated”

This line can sound strange, so let me explain carefully.

It does **not** mean brand-new money is randomly created every time.

It means:

when one or more old UTXOs are spent, the transaction creates **new output UTXOs**.

So old spendable pieces are consumed, and new spendable pieces are created.

13.4 Simple example

Suppose you have one UTXO worth 1000.

You want to pay someone 500.

Bitcoin transaction may:

- consume the old 1000 UTXO
- create one new UTXO of 500 to receiver
- create one change UTXO of remaining amount back to you
- possibly leave a fee difference for miner

So it is like breaking one old note into new notes.

That is a very good mental model.

14) Bitcoin legacy transaction format

The lecture then introduces the Bitcoin legacy transaction format. It shows the transaction has:

Input

- previous transaction ID
- index
- ScriptSig

Output

- value
- scriptPubKey

Also:

- lock_time
- value measured in satoshis
- examples of outputs to Babar and back to Alia as change.

This is a major exam topic.

14.1 Input

The input points to some previous unspent transaction output.

It tells the network:

“I want to spend that earlier output.”

To identify it, it uses:

- previous transaction ID
- index

Why index?

Because one transaction may have multiple outputs, so you must specify which output you are spending.

14.2 ScriptSig

This is the unlocking part from the spender's side.

It provides the signature and public key (or other unlocking data depending on script type).

14.3 Output

Each output creates a new spendable piece of value.

It contains:

- the value
- the locking script (scriptPubKey)

14.4 Lock_time

The lecture says lock_time may be 0 or current block number and is often ignored in simple cases.

This controls time-based validity behavior, though the course here is not going deep into it yet.

14.5 Change output

The page 21 visual is very helpful: Alia's input comes from an earlier unspent amount, and then two outputs are created — one to Babar and one back to Alia as change.

That diagram is conceptually excellent.

So remember:

Bitcoin spending often creates:

- payment output
 - change output
-

15) ScriptSig and ScriptPubKey

This is one of the most important Week 5 topics.

The lecture says:

ScriptSig

- encodes the signature and public key of the payer

ScriptPubKey

- sequence of instructions implementing a predicate / Boolean function
- takes ScriptSig as input
- returns True if input corresponds to legitimate owner, else False

If ScriptPubKey(ScriptSig) is true, then create output.

The output locks payment to hash(public key of payee).

This is called **Pay to PubKey Hash**.

Let's unpack this very carefully.

15.1 ScriptSig = unlocking script

ScriptSig is provided by the spender.

It basically says:

"Here is my evidence that I am allowed to spend this old output."

In these slides, that evidence includes:

- signature
- public key

15.2 ScriptPubKey = locking script

ScriptPubKey was already stored in the **previous output** being spent.

It contains the rule:

“What must someone provide to unlock me?”

So the old output says, in effect:

“To spend me, you must prove you own the corresponding key.”

15.3 Why both are needed

When a spender tries to use an old UTXO:

- ScriptSig provides candidate proof
- ScriptPubKey tests that proof

If it passes, spending is valid.

15.4 Boolean predicate idea

The lecture calls ScriptPubKey a Boolean function. That means its job is to evaluate to:

- True → valid owner proof
- False → invalid

So spending is basically script evaluation.

This is a really strong conceptual point.

16) P2PKH — Pay to PubKey Hash

The lecture then names the common legacy type:

Pay to PubKeyHash (P2PKH)

It says:

- payment is made to a public key
- public keys are large (64 bytes), while hash of pubkey is only 20 bytes
- so the output is locked to the hash of a public key
- only the payee can unlock the unspent bitcoin.

16.1 Why use hash of public key instead of raw public key?

Because hashes are smaller and convenient.

The lecture explicitly says:

- pubkey is 64 bytes
- pubkey hash is 20 bytes

So storing the hash is more compact.

16.2 What does it mean to lock to pubkey hash?

It means the output says:

“To spend me, provide a public key whose hash matches this stored hash, and provide a valid signature.”

So the recipient later proves ownership by:

- showing the correct public key
- showing valid signature

16.3 Why only payee can unlock?

Because only the payee should possess the private key corresponding to that public key hash.

So this is how Bitcoin encodes ownership control.

17) Actual Bitcoin transaction example

The lecture then shows a real transaction example with:

- transaction ID
- input referencing earlier transaction ID and index
- ScriptSig data
- two outputs
- values in BTC
- fees
- total input = outputs + fee
- one output to Babar and one output back to Alia.

This example is very useful conceptually.

17.1 What should you notice?

The important structure is:

- input comes from earlier transaction output
- outputs create new spendable outputs

- not all money goes to receiver
- some returns as change
- fee is the leftover difference

17.2 Why total input > one output?

Because the spender may consume one or more UTXOs larger than the payment amount.

So Bitcoin does not “cut” the old coin partially without structure.
It consumes inputs and creates new outputs.

17.3 Fee logic

Fee = total input – total outputs

This is a key relationship.

18) Stack-based execution — what the lecture is doing

Now the lecture spends many slides showing execution on a **stack**.

The goal is to check that the locked bitcoin in the previous transaction really belongs to Alia.
The stack diagrams gradually process:

- signature
 - public key
 - OP_DUP
 - OP_HASH160
 - expected pubkey hash
 - OP_EQUALVERIFY
 - OP_CHECKSIG
- and eventually produce **True**.

This is one of the most technical parts, so let's make it easy.

18.1 Big picture first

Bitcoin script executes like a little stack program.

The spender supplies ScriptSig.

The earlier output provides ScriptPubKey.

They are evaluated together.

If the final result is True, spending is allowed.

18.2 What is a stack?

A stack is a data structure where items are pushed and popped in last-in-first-out style.

You can think of it like a pile of plates:

- put something on top
- later take from top

Bitcoin script uses that style.

19) The P2PKH script in this lecture

The lecture shows the common ScriptPubKey operations:

- OP_DUP
- OP_HASH160
- <H(pubkey)>
- OP_EQUALVERIFY
- OP_CHECKSIG

And ScriptSig provides:

-

Let's explain exactly what each does.

19.1 ScriptSig pushes two things

From the spender side, ScriptSig places onto the stack:

- signature
- public key

So now the script has candidate unlocking data.

19.2 OP_DUP

Duplicates the public key on the stack.

Why?

Because one copy will be hashed and compared, while another will remain for signature checking.

19.3 OP_HASH160

Hashes the copied public key.

So now the script computes the spender-provided public key hash.

19.4 <H(pubkey)>

This is the expected public key hash that was already stored in the old output's ScriptPubKey.

So now the stack contains:

- computed hash from provided pubkey
- expected stored hash

19.5 OP_EQUALVERIFY

Checks whether those two hashes are equal.

If not equal, fail immediately.

If equal, continue.

This proves that the provided public key corresponds to the public key hash the coin was originally locked to.

19.6 OP_CHECKSIG

Now the script checks whether the provided signature is valid for the transaction under the provided public key.

If valid, result is True.

So this proves:

- correct owner key was provided
- valid authorization was given

That is why the final result on the slide becomes **True**.

20) The easiest way to understand stack execution

Here is the story version:

The old output says:

“I am locked to this pubkey hash. Only the rightful owner may spend me.”

Alia tries to spend it, so she provides:

- her public key
- her signature

The script then asks two questions:

Question 1:

Does the provided public key hash to the same hash that this coin was originally locked to?

If no → reject

If yes → continue

Question 2:

Does the signature verify under that public key for this transaction?

If no → reject

If yes → accept

That is literally the heart of Bitcoin output spending.

21) Why does the lecture say “transaction excluding signature”?

On page 40, the lecture shows signature verification conceptually as:

transaction excluding signature = payer || payee || value, and the signature is over the hash of that message. The page 40 diagram shows the signer’s public key, signature function, and message digest comparison.

This is reminding you that for signature verification, you do not sign the signature itself recursively.

Instead, you sign the transaction message content, and the verifier recomputes the digest and checks it.

So the lecture is tying Week 3 digital-signature logic into actual Bitcoin transaction verification.

22) Final result of execution

The page 41 slide shows the final stack result as **True**.

That means:

- ownership proof succeeded
- signature matched
- old locked bitcoin was legitimately spendable by Alia

So the input is valid.

Once the input is valid, the transaction can create the new outputs:

- one locked to Babar's pubkey hash
- one locked to Alia's pubkey hash as change. The final summary on page 43 explicitly says the outputs are created if the inputs are valid, and the outputs are locked to Babar's and Alia's public key hashes respectively.

That page is the perfect high-level end summary.

Week 5 — Full connected story

Now let me connect the whole week as one big story.

The week begins by broadening blockchain beyond coins. A token is a digital unit of value recorded on a blockchain, but unlike a coin, it does not have its own blockchain. Instead, it is a programmable asset built on top of another blockchain, usually through smart contracts. Tokens can represent many things: money, access rights, votes, loyalty points, ownership shares, or real-world assets. Stablecoins are one especially important token type because they are designed to represent fiat money on-chain.

Stablecoins like USDC and USDT try to stay near one dollar by combining reserve backing with issuance/redemption rules. When users send dollars to the issuer, tokens are minted; when they redeem tokens, those tokens are burned and dollars are returned. This convertibility is what

supports the peg, while arbitrage keeps market price near \$1. If price rises above \$1, traders mint and sell; if it falls below \$1, traders buy cheap and redeem. The lecture also distinguishes ARTs, which reference baskets of assets, from EMTs, which are pegged to one fiat currency and are more payment-oriented.

The second half of the week becomes much more technical and focuses on Bitcoin transactions. The lecture first explains the familiar account-based model, where the ledger stores balances and a valid transaction simply moves value from one account to another. Ethereum is presented as an example of this style. But this model is vulnerable to replay if the same signed transaction can be reused, so systems like Ethereum attach a sequence-like unique identifier to transactions and track that it is not reused. Bitcoin avoids replay and double-spending differently through the UTXO model.

In Bitcoin, value is not mainly tracked as account balances. Instead, transactions consume earlier unspent outputs and create new unspent outputs. A UTXO can be spent only once. So when Alia spends a previous output, that old output is consumed and new outputs are created, such as one to Babar and one back to Alia as change. Bitcoin legacy transaction format therefore contains inputs that reference previous transaction IDs and output indices, along with unlocking data in ScriptSig, and outputs that carry value and locking conditions in scriptPubKey.

Those locking and unlocking conditions are implemented through Bitcoin Script. In the common legacy P2PKH form, ScriptSig provides the spender's signature and public key, while ScriptPubKey contains instructions like OP_DUP, OP_HASH160, a stored pubkey hash, OP_EQUALVERIFY, and OP_CHECKSIG. Together they ask: does the provided public key hash to the expected owner hash, and does the signature verify correctly for this transaction? If yes, the script evaluates to True and the input is valid. Once valid inputs are established, the transaction's outputs are created and locked to the next owners' public key hashes. That is the key idea of Week 5: Bitcoin validates spending not by looking at account balances directly, but by checking whether a previously locked output may now be legitimately unlocked and replaced by new outputs.

Most important Week 5 topics to memorize

These are the highest-priority ones:

- token definition
- coin vs token
- what tokens can represent
- how tokens work through smart contracts
- stablecoin issuance and redemption
- arbitrage keeping stablecoin near \$1

- ART vs EMT
 - financial transaction elements
 - account-based model
 - replay attack
 - SEQ / unique identifier to stop replay
 - UTXO model
 - Bitcoin legacy transaction format
 - input vs output
 - previous transaction ID and index
 - ScriptSig
 - ScriptPubKey
 - P2PKH
 - stack-based execution
 - OP_DUP, OP_HASH160, OP_EQUALVERIFY, OP_CHECKSIG
 - change output
 - final “True” meaning valid spend
-

Ultra-short Week 5 revision

A token is a programmable digital unit of value recorded on a blockchain, built on top of an existing blockchain rather than having its own. Stablecoins are tokens representing fiat value and stay near \$1 through reserve backing, issuance/redemption, and arbitrage. ARTs reference multiple assets, while EMTs reference a single fiat currency. In transaction design, the account-based model stores balances directly and must prevent replay using unique transaction identifiers. Bitcoin instead uses the UTXO model, where transactions consume old unspent outputs and create new ones. Legacy Bitcoin transactions contain inputs pointing to previous transaction IDs and output indices, plus ScriptSig for unlocking; outputs contain value and ScriptPubKey for locking. In the common P2PKH form, the spender provides signature and public key, the script checks that the public key hash matches the stored hash, and then verifies the signature. If the script evaluates to True, the spend is valid and new outputs are created for the payee and change address.